

# Project 4 实验报告

姓名：陳日康 学号：22336049

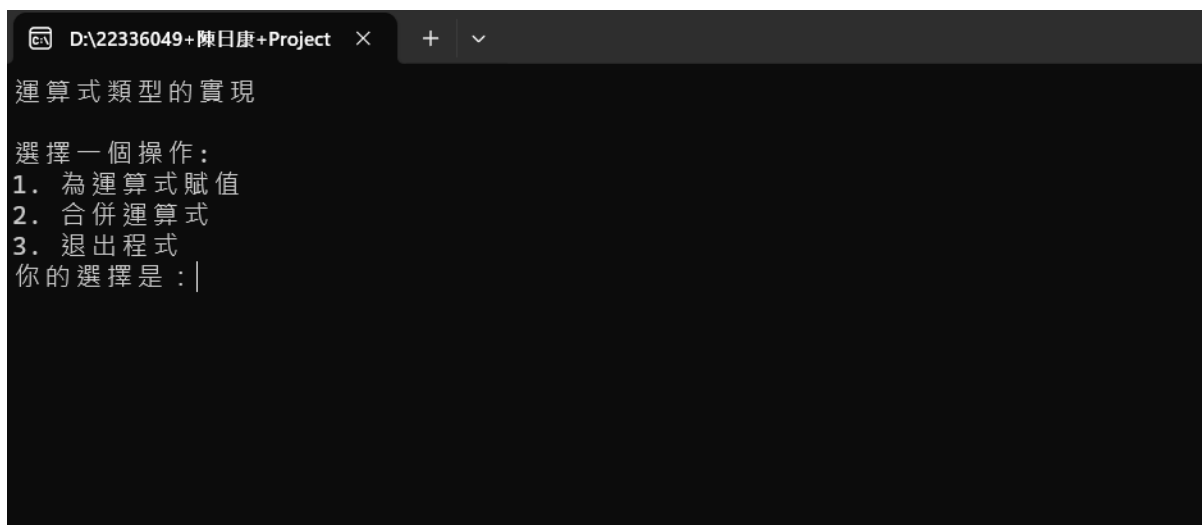
## 1、程序功能简要说明。

程序有三个功能：

1. 可以为表达式赋值：用户可以输入一个前缀算术表达式，然后根据需要为其中的变量赋值，程序将计算表达式的值并输出。
2. 合并表达式：用户可以输入两个前缀表达式，选择一个合并操作符（+、-、\*、/、^），然后程序将合并这两个表达式并输出合并后的中缀表达式。
3. 退出程序：用户可以选择退出程序。

程序循环运行，允许用户反复选择这三个操作中的一个。根据用户的输入，程序会相应地执行操作并输出结果。

## 2、程序运行截图，包括计算功能演示、部分实际运行结果展示、命令行或交互式界面效果等。



```
D:\22336049+陳日康+Project >
運算式類型的實現

選擇一個操作：
1. 為運算式賦值
2. 合併運算式
3. 退出程式
你的選擇是：|
```

首先，程序一开始会要求用户选择操作，选择 1 是为表达式赋值，选择 2 是合并表达式，选择 3 为退出程序。

```
D:\22336049+陳日康+Project  × + ▾
運算式類型的實現

選擇一個操作：
1. 為運算式賦值
2. 合併運算式
3. 退出程式
你的選擇是：1

為運算式賦值：
請輸入一個正確的前綴算術運算式：+a*bc
初始運算式為：(a+(b*c))

是否為變數賦值 (Y/N)? |
```

上图为部分运行结果，如果选择操作 1，程序会要求用户输入一个正确的前綴表达式，然后程序会计算出初始的中綴表达式，然后程序会让你选择是否为变量赋值（Y-是，N-否）。

```
D:\22336049+陳日康+Project  × + ▾
運算式類型的實現

選擇一個操作：
1. 為運算式賦值
2. 合併運算式
3. 退出程式
你的選擇是：1

為運算式賦值：
請輸入一個正確的前綴算術運算式：+a*bc
初始運算式為：(a+(b*c))

是否為變數賦值 (Y/N)? Y
請輸入變數和賦值，用空格分隔（例如：x 1）：a 1
變數 a 賦值為 1

是否繼續為變數賦值 (Y/N)? Y
請輸入變數和賦值，用空格分隔（例如：x 1）：b 1
變數 b 賦值為 1

是否繼續為變數賦值 (Y/N)? Y
請輸入變數和賦值，用空格分隔（例如：x 1）：c 1
變數 c 賦值為 1

是否繼續為變數賦值 (Y/N)? |
```

用户需要为变量赋值，每次均需选择 Y/N 操作，若选择 Y 操作，程序会要求用户为变量赋值，而输入格式为：变量 值（如：a 1）。

```
D:\22336049+陳日康+Project  ×  +  ∨

運算式類型的實現

選擇一個操作：
1. 為運算式賦值
2. 合併運算式
3. 退出程式
你的選擇是：1

為運算式賦值：
請輸入一個正確的前綴算術運算式：+a*bc
初始運算式為：(a+(b*c))

是否為變數賦值 (Y/N)? Y
請輸入變數和賦值，用空格分隔（例如：x 1）：a 1
變數 a 賦值為 1

是否繼續為變數賦值 (Y/N)? Y
請輸入變數和賦值，用空格分隔（例如：x 1）：b 1
變數 b 賦值為 1

是否繼續為變數賦值 (Y/N)? Y
請輸入變數和賦值，用空格分隔（例如：x 1）：c 1
變數 c 賦值為 1

是否繼續為變數賦值 (Y/N)? N

運算式的值為：2

-----
```

上图为部分运行结果，当用户不再为变量赋值时，可选择 N 操作停止赋值，而程序会算出最终赋值后表达式的结果。

```
D:\22336049+陳日康+Project  ×  +  ∨

變數 a 賦值為 1

是否繼續為變數賦值 (Y/N)? Y
請輸入變數和賦值，用空格分隔（例如：x 1）：b 1
變數 b 賦值為 1

是否繼續為變數賦值 (Y/N)? Y
請輸入變數和賦值，用空格分隔（例如：x 1）：c 1
變數 c 賦值為 1

是否繼續為變數賦值 (Y/N)? N

運算式的值為：2

-----

選擇一個操作：
1. 為運算式賦值
2. 合併運算式
3. 退出程式
你的選擇是：2

合併運算式：
輸入第一個前綴運算式：+a*bc
第一個前綴運算式中綴運算式：(a+(b*c))

輸入第二個前綴運算式：+*5^x2*8x
第二個前綴運算式中綴運算式：((5*(x^2))+(8*x))

輸入合併操作符 (+, -, *, /, ^): |
```

上图为部分运行结果。若用户选择操作 2，程序会要求用户先后输入两个前綴表达式，然后程序会计算出两个中綴表达式，然后用户需要选择合并的操作符 (+, -, \*, /, ^)。

```
D:\22336049+陳日康+Project x + v
是否繼續為變數賦值 (Y/N)? N

運算式的值為： 2

-----

選擇一個操作：
1. 為運算式賦值
2. 合併運算式
3. 退出程式
你的選擇是：2

合併運算式：
輸入第一個前綴運算式： +a*bc
第一個前綴運算式中綴運算式： (a+(b*c))

輸入第二個前綴運算式： ++5^x2*8x
第二個前綴運算式中綴運算式： ((5*(x^2))+(8*x))

輸入合併操作符 (+, -, *, /, ^): +

合併後的中綴運算式： ((a+(b*c))+((5*(x^2))+(8*x)))

-----

選擇一個操作：
1. 為運算式賦值
2. 合併運算式
3. 退出程式
你的選擇是：|
```

上图为部分运行结果，待用户选择需要的操作符后，程序会输出合并后的中綴表达式，这也是操作 2 的最终答案。

```
D:\22336049+陳日康+Project x + v
運算式類型的實現

選擇一個操作：
1. 為運算式賦值
2. 合併運算式
3. 退出程式
你的選擇是：3

已退出程序

-----
Process exited after 0.8515 seconds with return value 0
請按任意鍵繼續 . . . |
```

若然用户选择操作 3 退出程式，程序会输出“已退出程序”，这样程序就会结束。

```
D:\22336049+陳日康+Project × + v
運算式類型的實現

選擇一個操作：
1. 為運算式賦值
2. 合併運算式
3. 退出程式
你的選擇是：4

無效的選擇

-----

選擇一個操作：
1. 為運算式賦值
2. 合併運算式
3. 退出程式
你的選擇是：|
```

若然用戶輸入不符合的操作（1，2，3 以外），程序會輸出“無效的選擇”，然後用戶需要重新選擇操作。

測試數據：

操作1：

前綴表达式： $+a*bc$   
变量与赋值： $a=1, b=1, c=1$   
表达式的赋值结果：2

操作2：

第一个前綴表达式： $+a*bc$   
第二个前綴表达式： $+*5^x2*8x$   
合并的操作符： $+$   
合并后的结果： $(a+(b*c)) + ((5*(x^2)) + (8*x))$

经过对比，程序运行结果与计算结果一致。

### 3、部分关键代码及其说明。

```
9 struct BinaryNode {
10     char data;
11     BinaryNode* left;
12     BinaryNode* right;
13     BinaryNode(char d) {
14         data = d;
15         left = nullptr;
16         right = nullptr;
17     }
18 };
19
20 typedef BinaryNode* BinaryTree;
```

BinaryNode 结构体：

这是一个自定义结构体，用于表示二叉树的节点。char data 存储节点的数据，可以是操作符或操作数。BinaryNode\* left 是指向左子节点的指针。BinaryNode\* right 是指向右子节点的指针。构造函数 BinaryNode(char d) 用于创建节点对象，初始化数据并将左右子节点指针设置为 nullptr。

BinaryTree 别名：

这是一个类型别名，将 BinaryNode\* 简化为 BinaryTree，以表示二叉树的根节点。

```
23 BinaryTree ReadExpr (const string& expression, int& index) {
24     if (index >= expression.length ()) {
25         return nullptr;
26     }
27     char token = expression[index++];
28     if (islower (token)) {
29         return new BinaryNode(token);
30     }
31     else if (isdigit (token) || (token == '-' && isdigit(expression[index]))) {
32         return new BinaryNode(token);
33     }
34     else if (token == '+' || token == '-' || token == '*' || token == '/' || token == '^') {
35         BinaryTree left = ReadExpr (expression, index);
36         BinaryTree right = ReadExpr (expression, index);
37         BinaryNode* root = new BinaryNode (token);
38         root->left = left;
39         root->right = right;
40         return root;
41     }
42     else if (token == 'P') { // 新的操作符 P
43         BinaryTree E1 = ReadExpr (expression, index);
44         char P = expression [index++];
45         BinaryTree E2 = ReadExpr (expression, index);
46         BinaryNode* root = new BinaryNode (P);
47         root->left = E1;
48         root->right = E2;
49         return root;
50     }
51     return nullptr;
52 }
```

ReadExpr 函数：

BinaryTree ReadExpr (const string& expression, int& index) 用于解析前缀表达式并构建对应的二叉树。参数 expression 是前缀表达式的字符串。参数 index 是一个整数引用，用于跟踪读取位置。该函数采用递归方法，按顺序读取表达式字符，构建树的节点，并返回根节点指针。

```
56 void Assign (char V, int c) {
57     VarValue[V] = c;
58 }
59
60 int Power (int base, int exponent) {
61     if (exponent == 0) {
62         return 1;
63     }
64     return base * Power (base, exponent - 1);
65 }
```

Assign 函数：

void Assign (char V, int c) 用于将变量赋值为整数。参数 V 表示要赋值的变量。参数 c 是要分配给该变量的整数值。函数将变量名 V 与整数值 c 关联，存储在 VarValue 映射中。

Power 函数：

int Power (int base, int exponent) 计算整数的幂。参数 base 是底数，参数 exponent 是指数。使用递归方法计算 base 的 exponent 次幂并返回结果。

```
65 int Value (BinaryTree root) {
66     if (root) {
67         if (islower (root->data)) {
68             return VarValue[root->data];
69         }
70         else if (isdigit (root->data)) {
71             return root->data - '0';
72         }
73         else {
74             int leftVal = Value (root->left);
75             int rightVal = Value (root->right);
76             char op = root->data;
77
78             int result = 0;
79
80             switch (op) {
81                 case '+':
82                     result = leftVal + rightVal;
83                     break;
84                 case '-':
85                     result = leftVal - rightVal;
86                     break;
87                 case '*':
88                     result = leftVal * rightVal;
89                     break;
90                 case '/':
91                     if (rightVal != 0) {
92                         result = leftVal / rightVal;
93                     }
94                 else {
95                     cout << "除數不能為0" << endl;
96                     result = -1;
97                 }
98                 break;
99                 case '^':
100                     result = Power(leftVal, rightVal);
101                     break;
102             default:
103                 cout << "未知操作符: " << op << endl;
104                 result = -1;
105             }
106             return result;
107         }
108     }
109     return 0;
110 }
```

Value 函数：

int Value (BinaryTree root) 用于计算二叉树表示的表达式的值。参数 root 是指向二叉树根节点的指针。函数遍历二叉树，执行各种算术操作（加法、减法、乘法、除法、幂运算），并返回计算结果。

```

112 // 函數用於創建一個二叉樹節點
113 BinaryTree CreateNode (char data) {
114     return new BinaryNode(data);
115 }
116
117 // 函數用於輸出二叉樹的算術運算式
118 void PrintExpression (BinaryTree root) {
119     if (root) {
120         if (root->left || root->right) {
121             cout << "(";
122         }
123
124         PrintExpression (root->left);
125         cout << root->data;
126
127         PrintExpression (root->right);
128
129         if (root->left || root->right) {
130             cout << ")";
131         }
132     }
133 }
134
135 BinaryTree MergeExpressions (BinaryTree expr1, BinaryTree expr2, char op) {
136     BinaryNode* root = new BinaryNode (op);
137     root->left = expr1;
138     root->right = expr2;
139     return root;
140 }

```

CreateNode 函数：

BinaryTree CreateNode (char data) 用于创建一个新的二叉树节点。参数 data 是要存储在节点中的字符数据。函数返回指向新节点的指针。

PrintExpression 函数：

void PrintExpression (BinaryTree root) 用于以中缀形式输出二叉树表示的表达式。参数 root 是指向要输出的二叉树的根节点的指针。函数采用递归方式，添加括号以表示操作符的优先级。

MergeExpressions 函数：

BinaryTree MergeExpressions (BinaryTree expr1, BinaryTree expr2, char op) 用于合并两个二叉树表示的表达式。参数 expr1 和 expr2 分别是要合并的两个表达式的根节点。参数 op 是用作新根节点的操作符。函数创建一个新节点，将 expr1 和 expr2 作为左右子节点连接，并返回新的根节点。

#### 4、程序运行方式简要说明。

这是一个交互式前缀表达式计算程序，程序运行方式的详细说明如下：启动程序后，屏幕上将显示一个操作菜单，包括三个选项：1、为表达式赋值；2、合并表达式；3、退出程序。用户需要输入相应的数字来选择操作。如果用户输入的数字无效，程序将提示用户重新输入。程序在每个操作后会显示相应的输出，包括计算结果或合并的中缀表达式。用户可以根据提示选择要执行的操作，以与前缀表达式进行交互操作。



如果用户选择了操作 1，程序将要求用户输入一个正确的前缀算术表达式。用户输入表达式后，程序会显示该表达式的初始形式，并询问是否为变量赋值。如果用户选择赋值，程序将要求用户输入一个变量和相应的值。用户可以一次或多次进行变量赋值。在每次赋值后，程序将显示更新后的表达式，并询问用户是否继续赋值。用户可以选择继续或结束赋值操作。最终，程序将计算表达式的值，并输出结果。

如果用户选择了操作 2，程序将要求用户输入两个正确的前缀算术表达式。用户输入两个表达式后，程序将显示这两个表达式的初始形式，并询问用户输入一个合并操作符（+、-、\*、/、^）。程序将合并这两个表达式，形成一个新的中缀表达式，并输出结果。

如果用户选择了操作 3，程序将显示退出消息并结束运行。在用户输入不合法的操作或不正确的表达式时，程序将显示错误消息，并要求用户重新输入。错误消息将提供有关错误的详细信息，以帮助用户纠正输入。

完成一次操作后，程序将再次显示操作菜单，提示用户选择下一步的操作。这个过程将持续循环，直到用户选择退出程序。